Updated Nov 2024

FreeRTOS stack usage and stack overflow checking

Stack Usage

[Also see the uxTaskGetStackHighWaterMark() API function]

Each task maintains its own stack. If a task is created using xTaskCreate() then the memory used as the task's stack is allocated automatically from the FreeRTOS heap, and dimensioned by a parameter passed to the xTaskCreate() API function. If a task is created using xTaskCreateStatic() then the memory used as the task's stack is pre-allocated by the application writer. Stack overflow is a very common cause of application instability. FreeRTOS therefore provides two optional mechanisms that can be used to assist in the detection and correction of just such an occurrence. The option used is configured using the configCHECK_FOR_STACK_OVERFLOW configuration constant.

Note that these options are only available on architectures where the memory map is not segmented. Also, some processors could generate a fault or exception in response to a stack corruption before the RTOS kernel overflow check can occur. The application must provide a stack overflow hook function if configCHECK_FOR_STACK_OVERFLOW is not set to 0. The hook function must be called vApplicationStackOverflowHook(), and have the prototype below:

```
1  void vApplicationStackOverflowHook( TaskHandle_t xTask,
2                                      char *pcTaskName );
```

Copy

The xTask and pcTaskName parameters pass to the hook function the handle and name of the offending task respectively. Note however, depending on the severity of the overflow, these parameters could themselves be corrupted, in which case the pxCurrentTCB variable can be inspected directly.

Stack overflow checking introduces a context switch overhead so its use is only recommended during the development or testing phases.

Stack Overflow Detection - Method 1

It is likely that the stack will reach its greatest (deepest) value after the RTOS kernel has swapped the task out of the Running state because this is when the stack will contain the task context. At this point the RTOS kernel can check that the processor stack pointer remains within the valid stack space. The stack overflow hook function is called if the stack pointer contain a value that is outside of the valid stack range.

This method is quick but not guaranteed to catch all stack overflows. Set configCHECK_FOR_STACK_OVERFLOW to 1 to use this method.

Stack Overflow Detection - Method 2

When a task is first created its stack is filled with a known value. When swapping a task out of the Running state the RTOS kernel can check the last 16 bytes within the valid stack range to ensure that these known values have not been overwritten by the task or interrupt activity. The stack overflow hook function is called should any of these 16 bytes not remain at their initial value.

This method is less efficient than method one, but still fairly fast. It is very likely to catch stack overflows but is still not guaranteed to catch all overflows.

Set configCHECK_FOR_STACK_OVERFLOW to 2 to use this method.

Stack Overflow Detection - Method 3

Set configCHECK_FOR_STACK_OVERFLOW to 3 to use this method.

This method is available only for selected ports. When available, this method enables ISR stack checking. When an ISR stack overflow is detected, an assert is triggered. Note that the

stack overflow hook function is not called in this case because it is specific to a task stack and not the ISR stack.